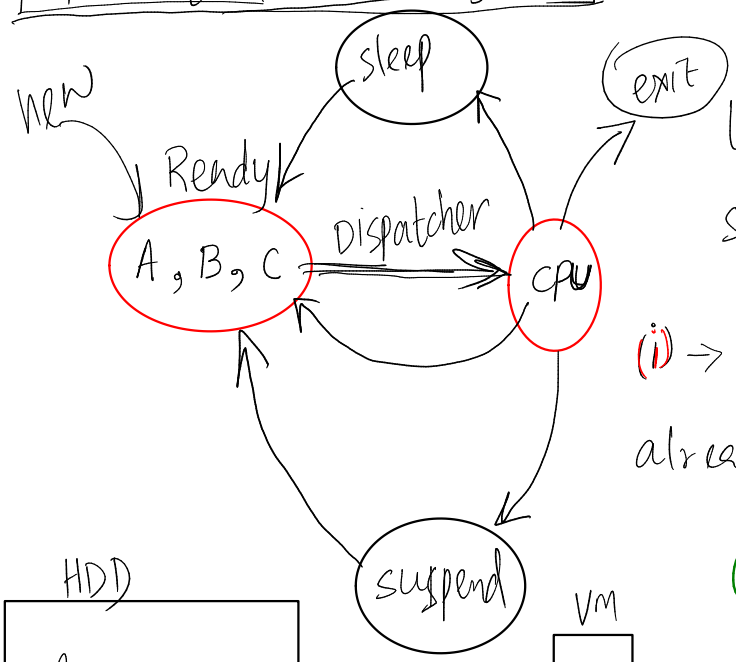
State-transition-Diagram

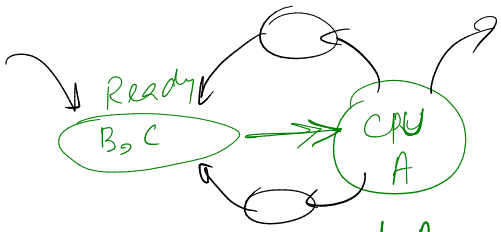


Ex. Three long jobs, are loaded.

lets assume Time slice in the RR-scheduling is 10ms.

(i) → A, B, C in Ready state, means, they're already in VM., May not be in RAM.

(ii) one job (A) is dispatched for execution



Demand Page occurs.

Page table used

Page fault. occured

NO Page fault

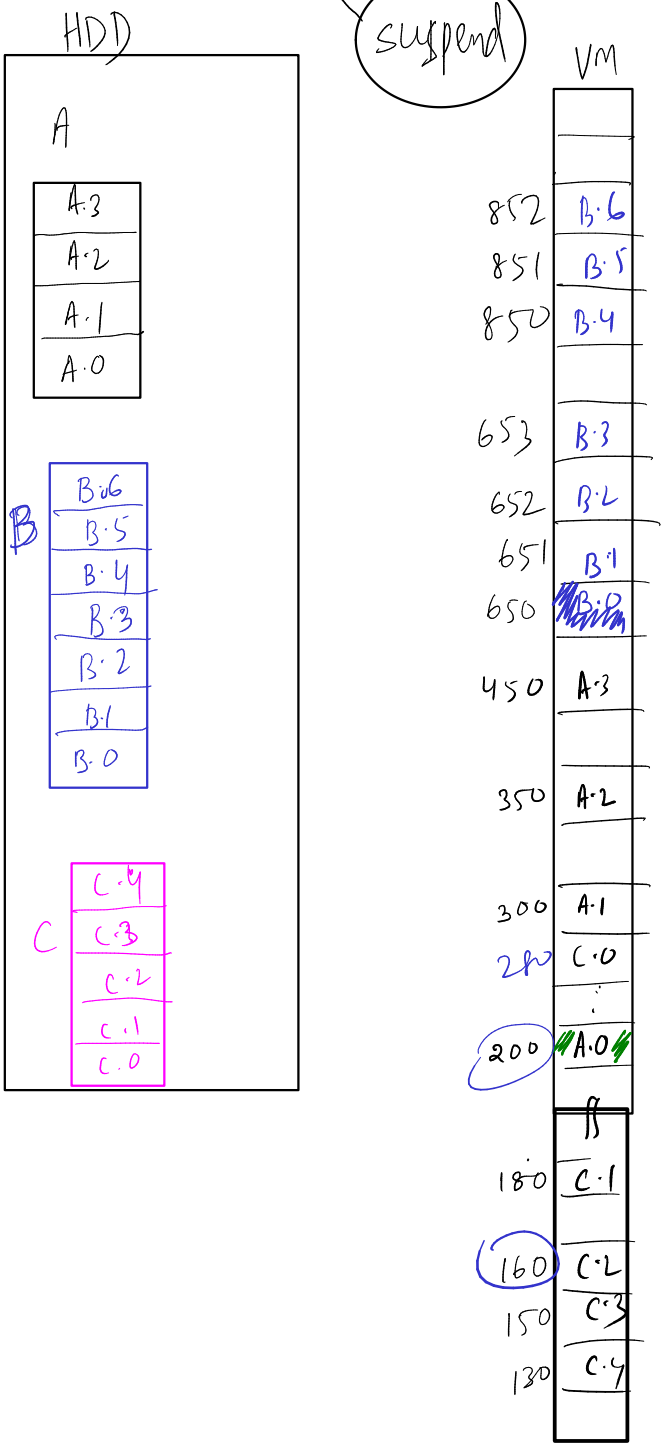
Page-IN

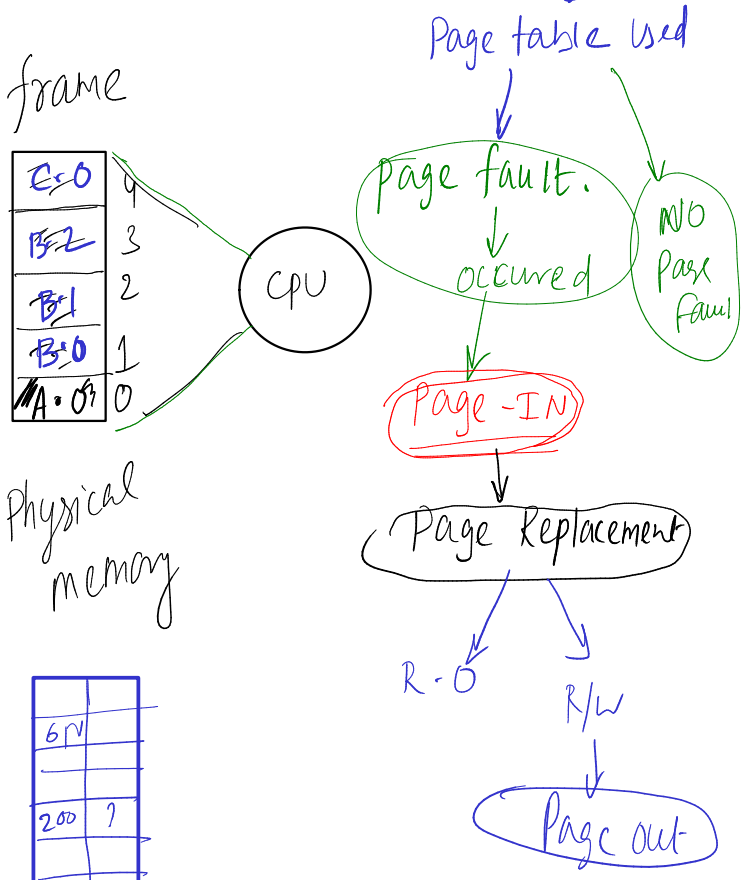
Page Replacement

R-O

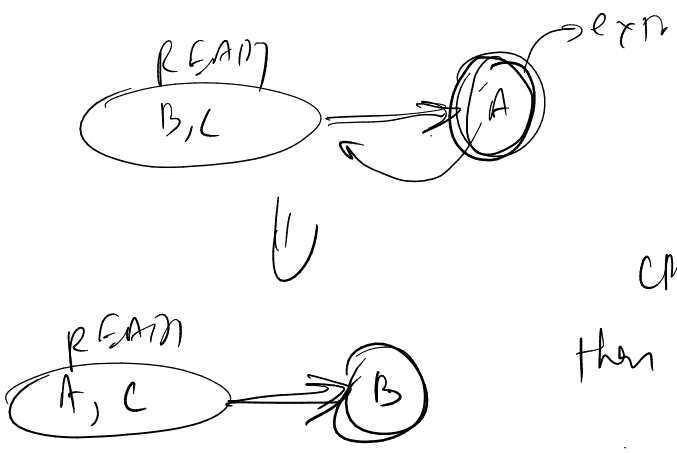
R/W

Page out





(iii) Time slice of A expires, Job A is out of CPU, dispatcher will ~~do~~ schedule another ready job.



Job switch/context switch.

~~PCB-A~~

CPU-context is stored into PCB-A, then B-PCB is loaded into CPU.

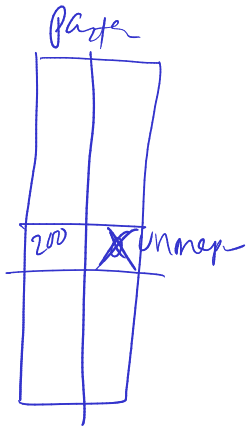
Triggering one Demand Paging

(iv) Job B T.S. completed, C needs to be dispatched

→ Demand Page occurs.

→ Page table -

→



(v) now C.2 is demanded

A.0 is replaced with C.2,

Frame 0 ~~A.0~~ C.2 (E)

Page X

Page-X is getting replaced by Page Y

Page-X will be paged out or not depends on page-X is Read-only or R-W

→ If pages are not contiguous, how kernel knows which page to be Mapped to Physical Mem

→ ~~If pages are contiguous, then~~

Exam-Paper 10 Questions - 60 mins

1 → 2 → 3 → 4 5 - - - - - - - 10

6min 6min

A new set with new Priority order

now 6 → 3 → 9 → 7 → 10 - - - -

and 10 → 91

Huge Paging Activity leads to thrashing

Process vs thread in Linux

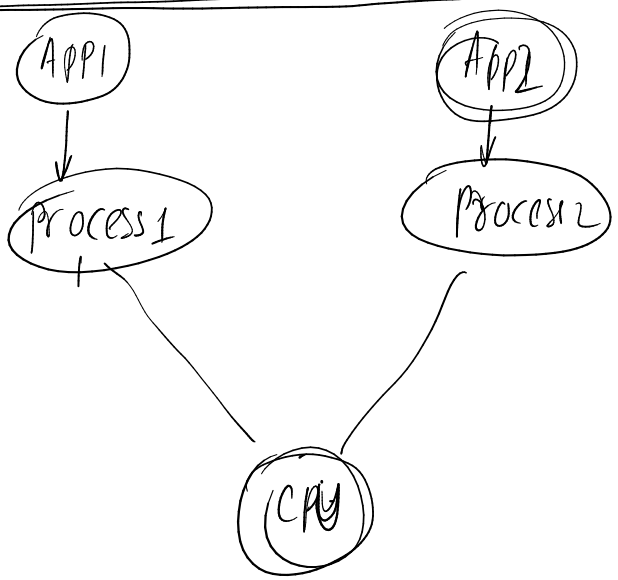
⇒ Every App on mem is a process.

⇒ Each process (out of N processes) has a separate Addr. Space (memory Area) pages

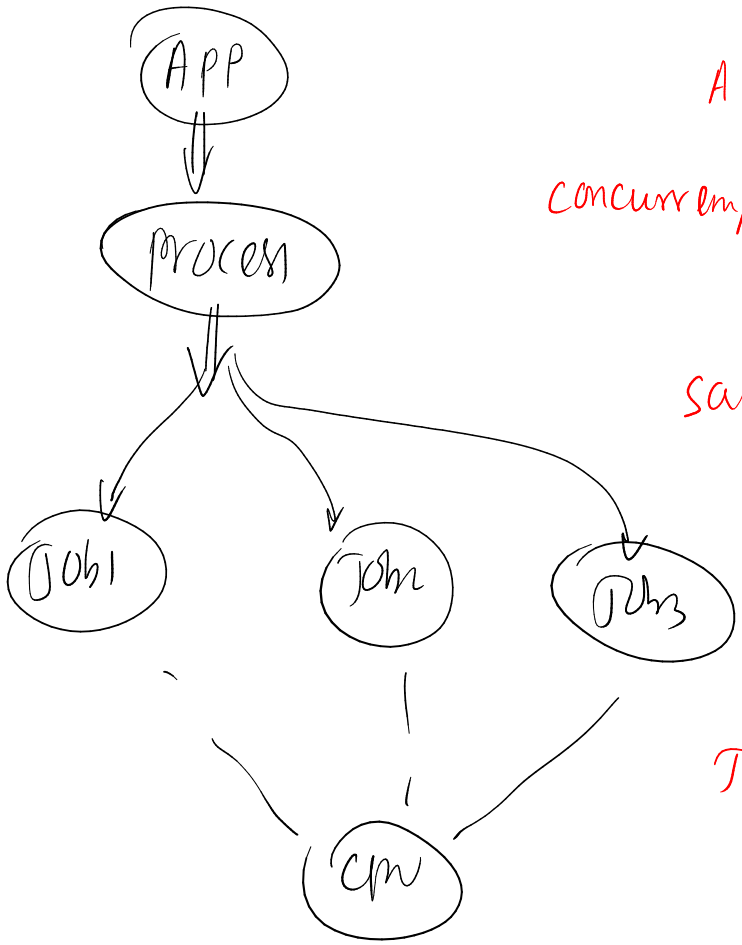
→ A process can't Access mem. Area beyond its allocated pages.

→ If tries to do so, then Segmentation-fault happens

↳ Mem-validation Interrupt



Each process has diff pagemaps / ~~App~~ process Area



A process, may have need of concurrency from within.

Job1, 2, 3 are created from same app/process.

Job 1, 2, 3 if processes then their Pagemap are Diffr

Job 1, 2, 3 if threads then their pages are common

threads switching is faster than process switching

→ Due to all jobs sharing common Addr Space

→ Page In-out will be relatively less